

14. Legibilidade de Código-Fonte

A legibilidade de código-fonte é a facilidade com que um programador consegue **ler, compreender e modificar um código**. Um código legível apresenta uma organização clara, nomes significativos, indentação adequada e uma estrutura consistente.

A legibilidade é importante porque facilita a manutenção do software, reduz a ocorrência de erros e permite que outros desenvolvedores compreendam o funcionamento do programa com mais facilidade.

14.1. Padrões de Nomenclatura

Os padrões de nomenclatura são regras utilizadas para definir como devem ser nomeados **variáveis, funções, classes, métodos e outros elementos do código**. O objetivo é tornar os nomes claros e padronizados.

Algumas boas práticas são:

- Utilizar nomes que indiquem claramente a finalidade do elemento.
- Evitar nomes muito curtos ou sem significado, como `x`, `a` ou `abc`, quando não forem apropriados ao contexto.
- Manter um padrão consistente durante todo o projeto.
- Utilizar nomes em um único idioma, conforme o padrão adotado pela equipe.
- Evitar abreviações desnecessárias.
- Utilizar convenções como **camelCase**, **PascalCase** ou **snake_case**, de acordo com a linguagem ou padrão do projeto.

Exemplo:

```
nome_cliente = "Carlos"
idade_cliente = 25
```

Nesse exemplo, os nomes permitem compreender facilmente o significado das variáveis.

14.2. Convenções de Linguagem

As convenções de linguagem são **regras e boas práticas específicas de uma determinada linguagem de programação**. Elas podem definir padrões para nomenclatura, organização do código, utilização de espaços, comentários, estrutura de funções e outros aspectos.

Seguir as convenções da linguagem torna o código mais uniforme e facilita sua leitura por outros programadores familiarizados com aquela tecnologia.

Por exemplo, em Python é comum utilizar **snake_case** para nomes de variáveis e funções:

```
def calcular_media(nota1, nota2):
    return (nota1 + nota2) / 2
```

Já em linguagens como Java e JavaScript, é comum encontrar o uso de **camelCase** para variáveis e métodos:

```
calcularMedia()  
nomeCliente  
valorTotal
```

O mais importante é seguir as convenções recomendadas para a linguagem e manter **consistência em todo o projeto**, evitando misturar diferentes padrões sem necessidade.